2019 年 11 月 12 日星期二

杭州信息中心社服系统处理服务器性能优化过程：

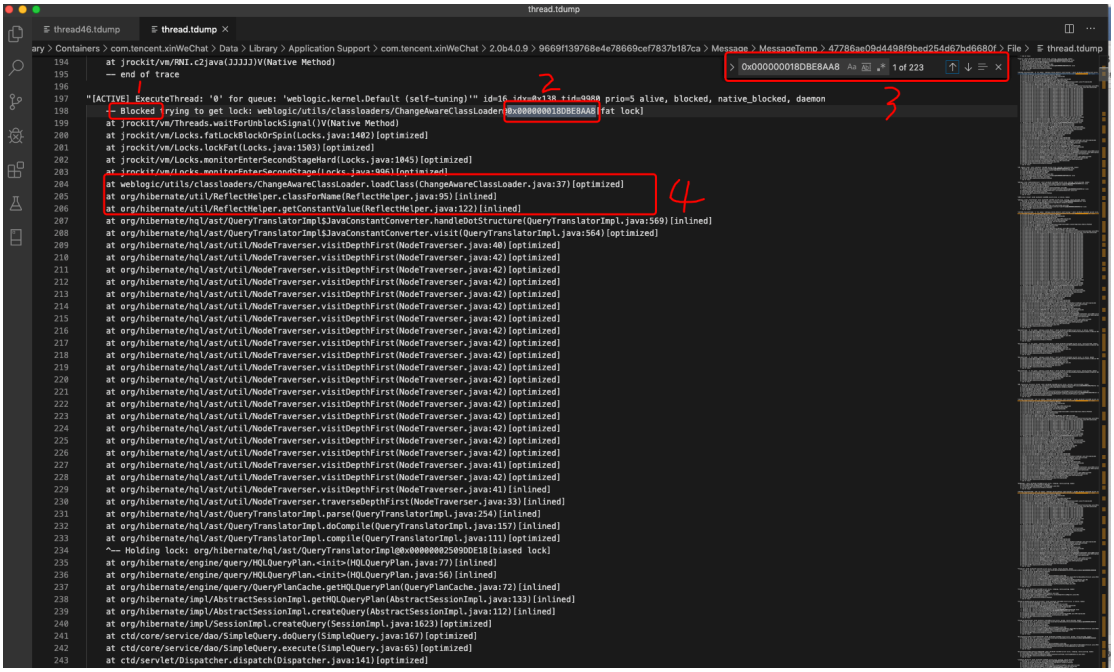
首先了解了系统的运行概况，利用 jvisualvm.exe 工具进行 dump,但是该工具 dump 的时候还会分析文件，所以速度较慢。在线上进行线程的运行状况查看之后，对线程进行 dump。单纯 dump 线程和堆信息的时候，用了一下命令:

Jstack 1000 >d:/thread.tdump
说明:1000 为 jvm 所在的进程 pid,d:/thread.tdump 为日志文件输出路径
dump 堆文件
jmap -dump:live,format=b,file=d://phis.hprof 1000
说明:1000 为 jvm 所在的进程 pid,d:/phis.hprof 为日志文件输出路径

若命令行输入以上命令时提示无此命令，请切换到{JAVA_HOME}/bin 目录下执行

问题一：

分析 dump 出来的线程信息如下：



其中 1 为阻塞的线程：一般线程出现问题都是 Blocked 状态，发现该阻塞线程获取的锁对象在这份 dump 线程中有 223 个，这是个不正常的现象，继续往下看，发现问题出现在 hibernate。

情况说明：

在 hibernate 5.2.6 版中，修改了这个问题，增加了配置
hibernate.query.conventional_java_constants
默认值为 true，即在 hql 中不支持 java 常量的引用

例如：
from Table t where t.c_bh=com.xxx.Table.N_XXX
不会尝试解析“t.c_bh”，但是会尝试解析“com.xxx.Table.N_XXX”
之前版本的 hibernate，在 hql 中碰到“xx.xx”时，都会调用 classloader 来尝试加载类，这个会导致 classloader 加载很多不存在的类（调用开销大），这是导致性能下降的原因。

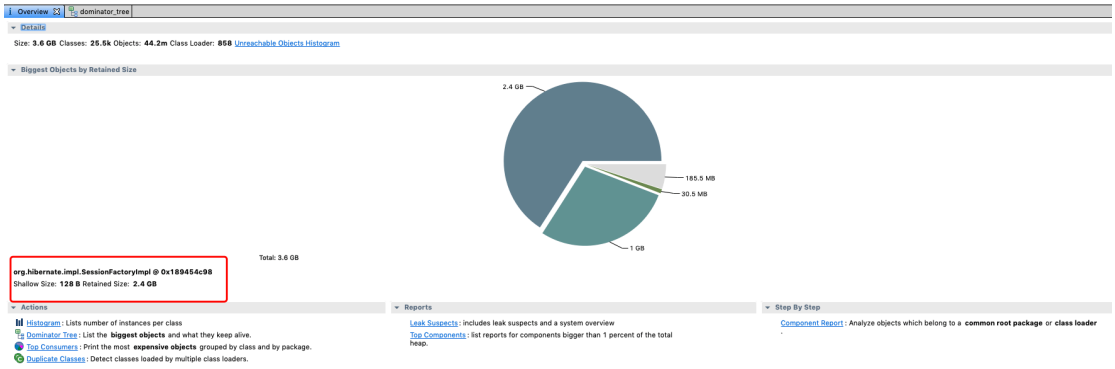
解决方案：

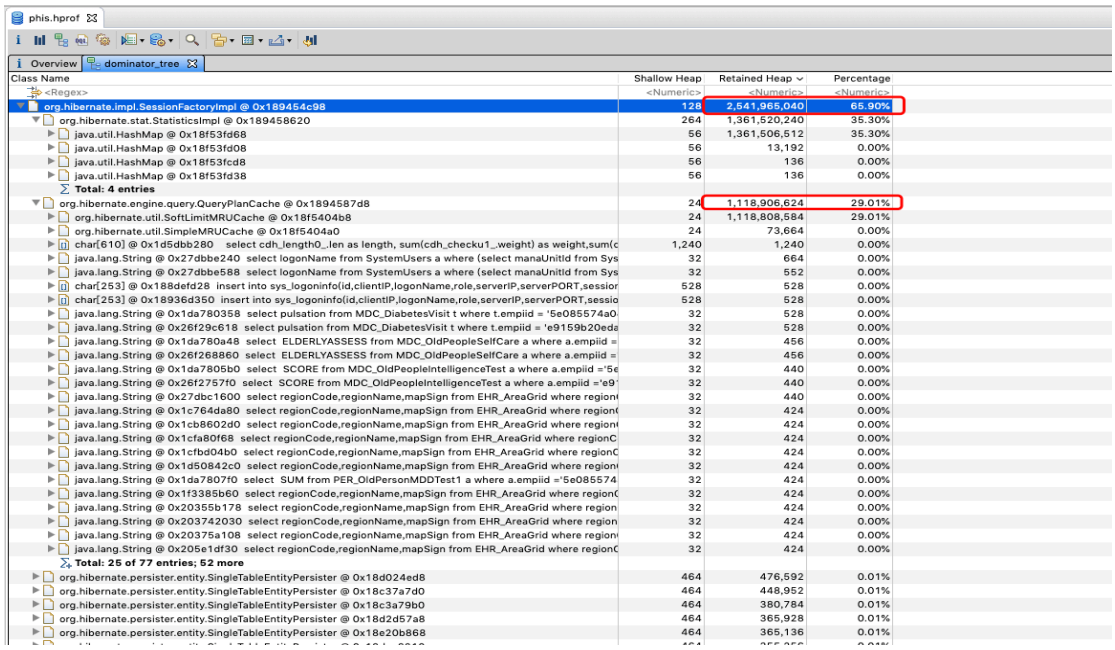
在高版本 hibernate 中,通过增加参数来控制是否启用常量解析功能来避免以上问题,但是因为整体升级 hibernate 版本对项目稳定性和兼容性存在一定风险,

所以可以选择重写 hibernate 相关类来达到相同目的.,重写了 getConstantValue,不进行常量的解析

问题二：

分析 dump 堆文件发现：





Class Name	Shallow Heap	Retained Heap	Percentage
<Regex>	<Numeric>	<Numeric>	<Numeric>
org.hibernate.impl.SessionFactoryImpl @ 0x189454c98	128	2,541,955,040	65.90%
org.hibernate.stat.StatisticsImpl @ 0x189458620	264	1,361,520,240	35.30%
java.util.HashMap @ 0x18f53fd68	56	1,361,506,512	35.30%
java.util.HashMap @ 0x18f53fd08	56	13,192	0.00%
java.util.HashMap @ 0x18f53fcd8	56	136	0.00%
java.util.HashMap @ 0x18f53fd38	56	136	0.00%
Total: 4 entries			
org.hibernate.engine.query.QueryPlanCache @ 0x1894587d8	24	1,118,906,624	29.01%
org.hibernate.util.SoftLimitMRUCache @ 0x18f5404b8	24	1,118,808,584	29.01%
org.hibernate.util.SimpleMRUCache @ 0x18f5404a0	24	73,664	0.00%
char[610] @ 0x1d5dbb280 select cdh_length0_.len as length, sum(cdh_checku1_.weight) as weight,sum(c	1,240	1,240	0.00%
java.lang.String @ 0x27dbbe240 select logonName from SystemUsers a where (select manaUnitId from Sys	32	664	0.00%
java.lang.String @ 0x27dbbe588 select logonName from SystemUsers a where (select manaUnitId from Sys	32	552	0.00%
char[253] @ 0x188defd28 insert into sys_logoninfo(id,clientIP,logonName,role,serverIP,serverPORT,session	528	528	0.00%
char[253] @ 0x18936d350 insert into sys_logoninfo(id,clientIP,logonName,role,serverIP,serverPORT,sessio	528	528	0.00%
java.lang.String @ 0x1da780358 select pulsation from MDC_DiabetesVisit t where t.empiid = '5e085574a0	32	528	0.00%
java.lang.String @ 0x26f29c618 select pulsation from MDC_DiabetesVisit t where t.empiid = 'e9159b20eda	32	528	0.00%
java.lang.String @ 0x1da780a48 select ELDERLYASSESS from MDC_OldPeopleSelfCare a where a.empiid =	32	456	0.00%
java.lang.String @ 0x26f268860 select ELDERLYASSESS from MDC_OldPeopleSelfCare a where a.empiid =	32	456	0.00%
java.lang.String @ 0x1da7805b0 select SCORE from MDC_OldPeopleIntelligenceTest a where a.empiid ='5e	32	440	0.00%
java.lang.String @ 0x26f2757f0 select SCORE from MDC_OldPeopleIntelligenceTest a where a.empiid ='e91	32	440	0.00%
java.lang.String @ 0x27dbc1600 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	440	0.00%
java.lang.String @ 0x1c764da80 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x1cb8602d0 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x1cfa80f68 select regionCode,regionName,mapSign from EHR_AreaGrid where regionC	32	424	0.00%
java.lang.String @ 0x1cfbd04b0 select regionCode,regionName,mapSign from EHR_AreaGrid where regionC	32	424	0.00%
java.lang.String @ 0x1d50842c0 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x1da7807f0 select SUM from PER_OldPersonMDDTest1 a where a.empiid ='5e085574	32	424	0.00%
java.lang.String @ 0x1f3385b60 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x20355b178 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x203742030 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x20375a108 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
java.lang.String @ 0x205e1df30 select regionCode,regionName,mapSign from EHR_AreaGrid where region	32	424	0.00%
Total: 25 of 77 entries; 52 more			
org.hibernate.persister.entity.SingleTableEntityPersister @ 0x18d024ed8	464	476,592	0.01%
org.hibernate.persister.entity.SingleTableEntityPersister @ 0x18c37a7d0	464	448,952	0.01%
org.hibernate.persister.entity.SingleTableEntityPersister @ 0x18c3a79b0	464	380,784	0.01%
org.hibernate.persister.entity.SingleTableEntityPersister @ 0x18d2d57a8	464	365,928	0.01%
org.hibernate.persister.entity.SingleTableEntityPersister @ 0x18e20b868	464	365,136	0.01%

情况说明:

实际占用者为 org.hibernate.stat.ConcurrentStatisticsImpl,自身占用内存高达 1.5G,且随着运行时长的增加还会不断增加.

查阅相关资料,发现该对象是用来存储统计查询缓存的监控信息,该功能是通过 hibernate 配置文件中的 hibernate.generate_statistics 参数控制,而发布版本默认的值为 true,即默认开启该功能.

而此缓存对象的数量过大说明查询的 sql 或 hql 语句的查询条件不是变量绑定或预编译的写法，在进行大量的动态 sql、hql 查询时就会导致此问题!而当前我们所使用的版本恰好存在大量的硬解析的数据库查询!

解决方案:

1.尽量使用预编译的 sql 或者 hql

2.将 spring-hibernate.xml 文件中的 hibernate.generate_statistics 设置为 false

问题三：

经过第一轮查证优化后，接着 dump 线程信息分析发现：

```
1       at java/lang/Thread.run(Thread.java:619)
2       at jrockit/vm/RNI.c2java(JJJJJ)V(Native Method)
3       -- end of trace
4
5   "[ACTIVE] ExecuteThread: '3' for queue: 'weblogic.kernel.Default [self-tuning]'" id=72 idx=0x200 tid=6888 prio=5 alive, blocked, native_blocked, daemon
6       -- Blocked trying to get lock: org/apache/log4j/spi/RootLogger@0x00000018D90BC68[thin lock]
7       at jrockit/vm/Locks.waitForThinRelease(Locks.java:934)[optimized]
8       at jrockit/vm/Locks.monitorEnterSecondStageHard(Locks.java:1074)[optimized]
9       at jrockit/vm/Locks.monitorEnterSecondStage(Locks.java:996)[optimized]
0       at org/apache/log4j/Category.callAppenders(Category.java:201)[optimized]
1       at org/apache/log4j/Category.forcedLog(Category.java:388)[inlined]
2       at org/apache/log4j/Category.log(Category.java:853)[optimized]
3       at org/apache/commons/logging/impl/Log4JLogger.debug(Log4JLogger.java:110)[inlined]
4       at org/hibernate/hql/ast/tree/FromReferenceNode.setResolved(FromReferenceNode.java:51)[inlined]
5       at org/hibernate/hql/ast/tree/IdentNode.resolve(IdentNode.java:78)[optimized]
6       at org/hibernate/hql/ast/tree/FromReferenceNode.resolve(FromReferenceNode.java:94)[inlined]
7       at org/hibernate/hql/ast/tree/FromReferenceNode.resolve(FromReferenceNode.java:90)[optimized]
8       at org/hibernate/hql/ast/HqlSqlWalker.resolve(HqlSqlWalker.java:728)[optimized]
9       at org/hibernate/hql/antlr/HqlSqlBaseWalker.propertyRef(HqlSqlBaseWalker.java:1105)[inlined]
0       at org/hibernate/hql/antlr/HqlSqlBaseWalker.propertyRefLhs(HqlSqlBaseWalker.java:4920)[optimized]
1       at org/hibernate/hql/antlr/HqlSqlBaseWalker.propertyRef(HqlSqlBaseWalker.java:1066)[optimized]
2       at org/hibernate/hql/antlr/HqlSqlBaseWalker.selectExpr(HqlSqlBaseWalker.java:1881)[optimized]
3       at org/hibernate/hql/antlr/HqlSqlBaseWalker.selectExprList(HqlSqlBaseWalker.java:1825)[optimized]
4       at org/hibernate/hql/antlr/HqlSqlBaseWalker.selectClause(HqlSqlBaseWalker.java:1394)[optimized]
5       at org/hibernate/hql/antlr/HqlSqlBaseWalker.query(HqlSqlBaseWalker.java:553)[inlined]
6       at org/hibernate/hql/antlr/HqlSqlBaseWalker.selectStatement(HqlSqlBaseWalker.java:281)[optimized]
7       at org/hibernate/hql/antlr/HqlSqlBaseWalker.statement(HqlSqlBaseWalker.java:229)[optimized]
8       at org/hibernate/hql/ast/QueryTranslatorImpl.analyze(QueryTranslatorImpl.java:228)[optimized]
```

情况说明：
其中 log4j 日志输出有问题，发现生产环境日志输出级别较低，导致大量的日志输出占用了线程等待输入导致线程阻塞，降低性能。

解决方案：

修改日志输出级别至 ERROR；